TÀI LIỆU ÔN TẬP

# Chọn mô hình và thuật toán học máy

Quy trình 7 bước · key point từng bước · so sánh thuật toán · thước đo · câu hỏi tự kiểm tra

## Nguồn tài liệu

| File | Tên | Nội dung chính |
|---|---|---|
| DL04a | Model Selection (40 trang; tiêu đề PDF: Classification – Basic Concepts) | Supervised learning, bias–variance, underfitting/overfitting, sai số huấn luyện và sai số tổng quát hóa, Occam's razor, thước đo (confusion matrix, accuracy, cost, precision/recall/F, kappa, ROC/AUC), holdout, k-fold, siêu tham số, khoảng tin cậy, so sánh mô hình, chọn đặc trưng |
| DL04b | Models and Algorithms 01 (64 trang) | AI và ML, phạm vi ML (data, task, model, algorithm), dữ liệu như vector/điểm, lời nguyền số chiều, các nhiệm vụ ML, phân loại và các bộ phân loại phổ biến: kNN, Perceptron/Logistic Regression, MLP, Naïve Bayes, Decision Tree, SVM, Boosting, Random Forest |
| DL04c | Models and Algorithms 02 (38 trang) | Hồi quy (tuyến tính, ridge/lasso, mạng nơ-ron, Gaussian Process), giảm chiều (PCA, ICA, autoencoder), phân cụm (K-Means, Mini-Batch, Mean Shift, GMM, Affinity Propagation, DBSCAN, hierarchical, BIRCH), model selection |

## Cách đọc tài liệu này

- Nội dung **không gắn nhãn** là lấy từ slide (đã đối chiếu).
- Nội dung gắn nhãn **[Mình]** là **phân tích, suy luận hoặc kiến thức ngoài slide** của người soạn. Đây là phần bạn nên cân nhắc kỹ nhất khi kiểm tra.
- Ký hiệu ⚠ đánh dấu **lỗi hoặc điểm lệch trong slide** mà mình phát hiện khi đối chiếu.
- Cuối mỗi bước có hộp **Tự kiểm tra**: nguồn đã đối chiếu, đâu là ý tác giả slide, đâu là phân tích của mình, mức chắc chắn và phần chưa xác minh.
- Các nhận định kiểu "thường", "nên" trong slide là quy tắc kinh nghiệm của giảng viên, không phải định lý.

## Bản đồ: mỗi bước dựa vào file nào

| # | Bước | DL04a | DL04b | DL04c |
|---|---|---|---|---|
| 1 | Xác định bài toán và nhiệm vụ ML | Bài toán học có giám sát (E, f, H, h) | AI/ML, data–task–model–algorithm, các kiểu học, các nhiệm vụ | Regression vs classification, clustering vs classification |
| 2 | Dữ liệu như vector, lời nguyền số chiều | — | Vector/điểm, high dimensions, curse | — |
| 3 | Chọn họ mô hình và thuật toán | — | kNN, Perceptron/LR, MLP, NB, DT, SVM, ensembles | Regression, giảm chiều, clustering |
| 4 | Kiểm soát độ phức tạp | Bias–variance, under/overfitting, Occam | Tuning vs evaluating, tham số điều khiển độ phức tạp | Regularisation (ridge/lasso, alpha) |
| 5 | Thước đo hiệu năng | Confusion matrix, accuracy, cost, P/R/F, kappa, ROC | — | Metrics and scoring (nhắc tên) |
| 6 | Ước lượng đáng tin cậy và so sánh mô hình | Holdout, k-fold, siêu tham số, CI, t-test | Train/validation/test | Model selection (sklearn) |
| 7 | Chọn đặc trưng và quy trình tổng | Feature selection, kết luận | — | Preprocessing (nhắc tên) |

# BƯỚC 1 Xác định bài toán và nhiệm vụ ML

Nguồn: DL04b (p3–7, 24–46), DL04c (p3–4, 19–20), DL04a (p3–5)

## 1.1. AI, Machine Learning và phạm vi của ML (DL04b)

- **AI** là khái niệm rộng: máy thực hiện tác vụ theo cách ta xem là "thông minh". **ML** là một ứng dụng hiện nay của AI, dựa trên ý tưởng cho máy truy cập dữ liệu và để chúng tự học.
- ML: khoa học khiến máy tính hành động mà không cần lập trình tường minh; nhánh của AI tập trung phát triển thuật toán và mô hình thống kê; hệ thống học từ dữ liệu, nhận ra mẫu hình và ra quyết định với ít can thiệp của con người; phương pháp phân tích dữ liệu tự động hóa việc xây dựng mô hình phân tích.
- Ứng dụng: xe tự lái, nhận dạng giọng nói, tìm kiếm web hiệu quả, hiểu biết tốt hơn về hệ gen người.
- **Năm 'bộ lạc' của ML** (Domingos, 2012, đọc từ hình): Symbolists (inverse deduction, logic), Connectionists (neural networks, gradient descent), Evolutionaries (genetic programs, genetic search), Bayesians (probabilistic inference, posterior probability), Analogizers (support vectors, margin).

**KEY POINT**

- ML liên quan tới một loạt lựa chọn của **bốn thành phần: data, task, model, algorithm**.
- **Model**: cơ sở lý thuyết mà việc phân loại dựa vào. **Algorithm**: cách cụ thể ta thực hiện. Hai từ hay dùng lẫn lộn, nhưng cần phân biệt: (1) **model được học như thế nào**, (2) **model được dùng như thế nào**. Hai việc này hay bị gộp vì (2) có mặt trong (1).

**PHÂN TÍCH CỦA MÌNH (không có trong slide)**

- Ví dụ minh họa (mình đặt): kNN là model/ý tưởng 'dự đoán theo láng giềng gần nhất'. Algorithm là cách tìm láng giềng nhanh (như slide kNN nói: 'different algorithms for finding the nearest').

## 1.2. Mức giám sát và các nhiệm vụ chính

| Kiểu học | Mức giám sát | Mô tả (DL04b) |
|---|---|---|
| **Supervised** | Nhiều | Có 'người thầy' cho đáp án đúng ở một số trạng thái; mục tiêu: xuất đáp án đúng, 'do what the teacher would do'. |
| **Unsupervised** | Không | Không có thầy; hệ thống có 'người thầy' nội bộ với mục tiêu định sẵn, không cần phản hồi tiện ích. |
| **Reinforcement** | Một phần | Không có nhãn, môi trường cho **thưởng hoặc phạt** để chỉ ra tiện ích của hành động đã làm. |

| | Supervised | Unsupervised |
|---|---|---|
| Đầu ra rời rạc | **Classification** (phân loại) | **Clustering** (phân cụm) |
| Đầu ra liên tục | **Regression** (hồi quy) | **Dimension reduction** (giảm chiều) |

- Slide DL04b hỏi thêm: còn **semi-supervised learning** thì sao? (không đáp án). Ngoài ra reinforcement learning được 'để sang một bên' trong sơ đồ.
- Loại đầu vào: có thể là **i.i.d.** (độc lập, cùng phân phối), hoặc **chuỗi** (sequence), hoặc loại khác.
- Ontology của MATLAB (hình): Machine learning → Supervised (classification, regression) và Unsupervised (clustering); slide nhận xét 'it's a lot richer than this'.
- Đầu ra (outcome) có thể đơn giản hoặc giàu thông tin hơn: bộ phân loại trả lời một lớp, nhưng có thể coi là một 'dự đoán' kiểu nào đó.

## 1.3. Bốn nhiệm vụ

| Nhiệm vụ | Mục tiêu | Ví dụ trong slide | Ghi chú |
|---|---|---|---|
| **Classification** | Dự đoán một danh mục (lớp) | Iris: phân loại 3 loài (setosa, versicolor, virginica) từ 4 thuộc tính; khối u lành/ác; giao dịch thẻ hợp lệ/gian lận; phân loại tin tức | DL04a: x là email, f(x) là spam/ham |
| **Regression** | Dự đoán một hoặc nhiều số thực | x là ngôi nhà, f(x) là giá bán | Đầu ra liên tục so với nhãn rời rạc của classification (DL04c) |
| **Clustering** | Tìm các nhóm đối tượng 'tương tự' | Cùng dữ liệu Iris nhưng không dùng nhãn | Không giám sát: lớp của mẫu không biết. Thành công thường đo **chủ quan**: về bản chất là bài toán **ill-posed**. |
| **Dimension reduction** | Tìm các hướng chính trong dữ liệu | PCA và các phương pháp khác | Dữ liệu thô thường nhiều chiều: (1) con người khó diễn giải > 3 chiều; (2) thuật toán ML cũng gặp khó, về cuối là do curse of dimensionality. |

### Bài toán học có giám sát được phát biểu thế nào (DL04a)

- Ví dụ: cặp vào–ra E = $((x_1,y_1), ..., (x_N,y_N))$, giả định được sinh **iid** (có nhiễu và lỗi) từ hàm đích y = f(x).
- Bài toán học: cho không gian giả thuyết H, tìm h ∈ H sao cho $\hat{y}_i = h(x_i) \approx y_i$. Tức xấp xỉ f bằng h dùng E.
- Slide: 'You already know linear regression. We focus on Classification.'

### Classification và regression (DL04c): khác nhau ở đâu?

| Tiêu chí | Clustering | Classification |
|---|---|---|
| Số lớp | Chưa biết | Đã biết |
| Dữ liệu huấn luyện | Không bắt buộc | Bắt buộc |
| Mục đích | Làm việc trên dữ liệu hiện có | Phân loại các mẫu tương lai vào lớp |

Regression: đầu ra là một đại lượng liên tục. Classification: đầu ra là nhãn rời rạc. Regression dùng rộng rãi để dự đoán và mô tả quan hệ giữa các biến độc lập $X_i$ với biến phụ thuộc Y.

## 1.4. Classification: hai mục đích và các khía cạnh (DL04b)

| Mục đích | Nội dung |
|---|---|
| **Prediction** | Từ dữ liệu quá khứ xây mô hình để phân loại quan sát mới (chưa biết nhãn). Ước lượng 'giá trị' mô hình (ví dụ accuracy); **tập test độc lập với tập train** (nếu không sẽ overfitting); tập dùng để chọn/tinh chỉnh mô hình gọi là **validation set**. |
| **Insight** | Nhận ra đặc trưng dư thừa/không liên quan, mức quan trọng và quan hệ giữa các đặc trưng để hiểu đối tượng. Mô hình có thể là luật, cây quyết định, mạng... |

- **Định nghĩa:** không gian tìm kiếm S là tập mọi thể hiện có thể (ví dụ không gian giá xe và thu nhập); lớp C cần mô hình hóa (ví dụ 'xe gia đình'); **decision boundary** ngăn cách các vùng.
- Các khía cạnh (hình, theo sách Alpaydin 'Introduction to Machine Learning'): instances và classes; **errors**; **specific vs general** hypothesis; **binary vs multiclass**; đầu ra là **decision boundaries/regions** tức một giả thuyết, **tuyến tính hoặc phi tuyến**.

### TỰ KIỂM TRA CỦA BƯỚC NÀY

- **Đối chiếu nguồn:** DL04b trang 3–7, 24–36, 39–46; DL04c trang 3–4, 19–20; DL04a trang 3–5. Đã xem ảnh DL04b trang 6 (năm bộ lạc), 23 (ontology MATLAB). Các hình DL04b 42–46 (từ sách) mình chỉ có chữ.
- **Ý tác giả và của mình:** toàn bộ định nghĩa là của slide. Ví dụ kNN minh họa model/algorithm là của mình (đã gắn nhãn).
- **Bịa đặt/thiên kiến:** không thêm khái niệm ngoài slide. Slide DL04b bỏ ngỏ vài câu hỏi (semi-supervised); mình không tự trả lời.
- **Chắc chắn:** cao cho định nghĩa. Chưa xác minh: các hình minh họa classification (decision boundary, errors, specific/general) vì phần lớn là hình trích từ sách.

BƯỚC 2

# Dữ liệu như vector và lời nguyền số chiều

Nguồn: DL04b (p8–15)

## 2.1. Dữ liệu và công cụ (DL04b)

- Nguồn dữ liệu công khai: datahub.io, openml.org (ví dụ iris, penguins, diabetes, banknotes).
- Công cụ gần như chung: **python, numpy, sklearn, pandas, matplotlib, jupyter notebooks**.
- Ví dụ **banknotes.csv**: V1–V4 là bốn đặc trưng; Class = 1 (thật) hoặc 2 (giả); khoảng 1300 dòng, các dòng Class = 2 nằm phía dưới. Thường nói riêng về X (đặc trưng) và Y (nhãn).

## 2.2. Dữ liệu như vector trong không gian

- Mỗi dòng là một mục dữ liệu, gồm một cặp **X → y**. X = ($x_0$, $x_1$, $x_2$, $x_3$) là vector 4 chiều; $x_i$ là giá trị ở chiều thứ i.
- Mỗi dòng cũng có thể coi là một **điểm** trong một **không gian** dữ liệu. Dễ hình dung ở 1, 2, 3 chiều; con người không nhìn được hơn 3 chiều; **một số trực giác còn đúng, một số sai**.

## 2.3. Dữ liệu nhiều chiều: 'tất cả nằm ở các góc'

**KEY POINT**

- Nói đại khái ở nhiều chiều: (1) mọi thứ **ở các góc**; (2) mọi thứ **vuông góc nhau**; (3) mọi thứ **cách đều nhau**.

**Thí nghiệm của slide:** sinh một triệu điểm d chiều, mỗi chiều chọn ngẫu nhiên trong [−1, +1] (tức một 'hộp' cạnh 2). Quả cầu đường kính 2 nằm vừa khít trong hộp. Trong một triệu điểm có bao nhiêu điểm rơi vào quả cầu? Kết quả slide (từ hình):

| d | Số điểm trong cầu (slide) | Kỳ vọng lý thuyết (mình tính) |
|---|---|---|
| 1 | 1.000.000 | 1.000.000 |
| 2 | 785.776 | 785.398 |
| 3 | 523.488 | 523.599 |
| 5 | 164.873 | 164.493 |
| 10 | 2.581 | 2.490 |
| 15 | 11 | 12 |
| 20 | 0 | 0 |

**PHÂN TÍCH CỦA MÌNH (không có trong slide)**

- Cột lý thuyết là kỳ vọng: tỉ lệ thể tích quả cầu đơn vị so với hộp $[-1,1]^d$, nhân một triệu. Số của slide (mô phỏng ngẫu nhiên) khớp trong phạm vi sai số lấy mẫu (ví dụ d = 10: 2581 so với 2490, lệch khoảng 1,8 độ lệch chuẩn). Slide chỉ nêu kết quả, không nêu công thức.
- Ý nghĩa: khi d tăng, hầu như mọi điểm rơi ra ngoài quả cầu tức nằm ở 'các góc' của hộp. Tới d = 18 trở đi slide đếm được 0 điểm.

## 2.4. The Curse of Dimensionality (DL04b)

- Xét không gian vector giới hạn trong [0,1] mỗi chiều. Giả sử ta 'nén thô bạo' một hàm bằng cách **cho đúng một giá trị mỗi 'góc phần tư'**. Ở 2 chiều cần 4 số, ở 3 chiều cần 8 ô.
- Câu hỏi của slide: **ở d chiều cần bao nhiêu số?** (không đáp án trong slide).

**PHÂN TÍCH CỦA MÌNH (không có trong slide)**

- Đáp án mình suy ra từ quy luật 4 (2 chiều), 8 (3 chiều): cần $\mathbf{2^d}$ số. Ví dụ d = 20 đã cần hơn một triệu số ($2^{20}$ = 1.048.576). Đây là lý do cần rất nhiều dữ liệu khi số chiều tăng, và là lý do dùng giảm chiều/chọn đặc trưng (liên hệ DL03, DL04b p34).

### TỰ KIỂM TRA CỦA BƯỚC NÀY

- **Đối chiếu nguồn:** DL04b trang 8–15; đã xem ảnh trang 14, 15 (bảng đếm điểm và hình quadrant). Kiểm bằng code: kỳ vọng lý thuyết cho d = 1, 2, 3, 5, 10, 15, 20.
- **Ý tác giả và của mình:** thí nghiệm và nhận định 'ở các góc, vuông góc, cách đều' là của slide. Công thức lý thuyết và đáp án $2^d$ là của mình.
- **Chắc chắn:** cao cho bảng đếm (đọc từ chữ trong hình). Trung bình cho nhận định 'vuông góc nhau, cách đều nhau': slide nêu 'roughly speaking' và không chứng minh; mình không kiểm chứng riêng hai ý này.
- **Lưu ý:** số hàng '~1300 rows' của banknotes là con số slide ghi; mình không đối chiếu với bộ dữ liệu gốc.

**BƯỚC 3**

# Chọn họ mô hình và thuật toán

Nguồn: DL04b (p47–64), DL04c (p4–38)

**KEY POINT**

- Mỗi mô hình/thuật toán đi kèm **các lựa chọn (options)**; ví dụ kNN: k, 'gần' theo nghĩa nào (model), 'tìm gần' nhanh thế nào (algorithm).
- Mỗi họ có **tham số điều khiển độ phức tạp** riêng (xem bước 4): k, C, alpha, var_smoothing/Laplace, max_depth.
- Slide giới thiệu **nhanh (overview only)**; các hình minh họa lấy từ scikit-learn.

## 3A. Các bộ phân loại phổ biến (DL04b)

| Bộ phân loại | Ý tưởng | Điều khiển độ phức tạp (sklearn) | Ghi chú của slide |
|---|---|---|---|
| **k-Nearest Neighbours (kNN)** | Dự đoán dựa trên láng giềng gần nhất trong tập huấn luyện. Hình Iris: k = 1 cho phân hoạch Voronoi; k = 3; k = 50. | **k**: thấp → chỉ bắt láng giềng gần nhất, có thể **overfit**; cao → lấy trung bình nhiều láng giềng, có thể **underfit**. | 'Hard to beat!', nhưng 'can get slow…'. Có các thuật toán khác nhau để tìm láng giềng gần. |
| **Perceptron / Logistic Regression** | Ngưỡng 'mềm' (soft threshold) cho phép 'nghi ngờ'; tương ứng vẽ một **siêu phẳng** trong không gian vector đầu vào x. Học = đổi trọng số, làm siêu phẳng dịch chuyển. | **Regularisation**: phạt kích thước trọng số, buộc đánh đổi lỗi và trọng số lớn. sklearn LogisticRegression gọi là **C** (nghịch đảo độ mạnh regularization; C nhỏ → regularization mạnh). | Slide ghi 'also known as logistic regression' và nêu nhiều lựa chọn huấn luyện. |
| **Multi-layer Perceptron (MLP)** | Xâu chuỗi các perceptron thành mạng nơ-ron; tổng quát hóa ranh giới siêu phẳng thành ranh giới **phức tạp hơn**. | Thêm số hạng phạt (regularisation) để trọng số nhỏ (ranh giới trơn hơn). sklearn gọi là **alpha** (độ mạnh số hạng L2). | |
| **Naïve Bayes** | Mô hình **sinh (generative)**: tưởng tượng lớp Y 'tạo ra' X. Giả định Naïve Bayes: Y tạo mỗi $X_i$ **độc lập** với các X khác → học dễ. Xác định Prob($X_i$ \| Y) riêng cho từng i từ tập huấn luyện; lúc test dùng Bayes để tính Prob(Y \| X mới). | Multinomial: **smoothing** (pseudo-count, Laplace smoothing) tránh overfit với đếm bằng 0. Gaussian: fit một Gaussian cho mỗi Prob($X_i$ \| Y); đặt cận dưới cho phương sai: **var_smoothing**. | 'Trông quá đơn giản' nhưng hiệu quả cho phân loại văn bản, lọc spam. Hai vị: **Multinomial** nếu $X_i$ là danh mục; **Gaussian** nếu $X_i$ là số thực. Bề mặt quyết định cũng là **siêu phẳng**, nhưng đặt khác perceptron. |
| **Decision Tree** | Hỏi lần lượt các câu như 'V2 > 5.1?', rồi đệ quy; lá của cây là kết quả phân loại. Có thể học: biến nào xét theo thứ tự, và ngưỡng. | sklearn gợi ý **max_depth**. | |
| **SVM / Kernel machines** | Hai cách nghĩ: (1) như kNN, có một hàm **kernel** đo 'độ gần' giữa hai ví dụ; (2) như perceptron, vẽ siêu phẳng trong một không gian. Nhờ 'mẹo toán học', không gian rất nhiều chiều này **không bao giờ được dựng tường minh** và siêu phẳng cũng không được tính tường minh. | (Slide không nêu tham số.) | Rất thành công; **khá bền với lựa chọn hàm kernel**. **Kernel trick**: ranh giới quyết định tuyến tính ở không gian chiều cao nhưng phi tuyến ở không gian đầu vào. |
| **Ensembles**: Boosting, Random Forests | Nhiều bộ phân loại hàng đầu dựa trên tập hợp bộ phân loại đơn giản: hàng trăm bộ phân loại 'cơ sở' có thể rất yếu (ví dụ **decision stump**, cây một tầng) rồi gộp dự đoán. | Như Decision Tree: **max_depth**. | |

### Gradient Boosted Decision Tree và Random Forests

- **GBDT**: dựng tập hợp cây quyết định **lần lượt từng cây**; dự đoán của các cây được **cộng** lại; cây kế tiếp cố gắng **bù phần chênh lệch** giữa giá trị đích và dự đoán hiện tại của tập hợp.
- **Random Forests**: mỗi cây được dựng từ một **bootstrap sample** (mẫu lấy **có hoàn lại**) của tập huấn luyện; khi tách mỗi nút, điểm tách tốt nhất được tìm hoặc từ **tất cả đặc trưng** hoặc từ một **tập con ngẫu nhiên** các đặc trưng.

**⚠ ĐIỂM CẦN LƯU Ý**

- Slide DL04b dùng tên 'Perceptron classifier (Logistic Regression)', coi hai thứ là một. Hai khái niệm này có liên quan nhưng không đồng nhất: slide mô tả 'ngưỡng mềm cho phép nghi ngờ', tức hành vi của logistic regression (đầu ra là xác suất), khác perceptron cổ điển với ngưỡng cứng. Khi thi, theo cách gọi của slide.

**PHÂN TÍCH CỦA MÌNH (không có trong slide)**

- Phần 'Naïve Bayes: hai mô hình khác nhau cùng bề mặt quyết định là siêu phẳng' được slide gọi là 'remarkable factoid', và nhấn mạnh **hai mô hình sẽ đặt siêu phẳng khác nhau**. Ý nghĩa: cùng dạng ranh giới không có nghĩa cùng mô hình.

## 3B. Hồi quy (DL04c)

| Mô hình | Ý tưởng | Điều khiển độ phức tạp / ghi chú |
|---|---|---|
| **Simple linear regression** | $y_i = \alpha + \beta x_i + \varepsilon_i$. Học độ dốc và hệ số chặn: khởi tạo ngẫu nhiên, 'loss function' (cost) là **tổng bình phương sai số**, dùng bộ tối ưu để cực tiểu hóa. | Đơn giản nhưng hữu ích, **dễ diễn giải**. |
| **Multiple linear regression** | Ở 1 chiều: đường thẳng; nhiều chiều: **siêu phẳng**. Một hệ số chặn, nhiều độ dốc (hệ số/trọng số): $y_i = \alpha + \Sigma\, \beta_k x_{k,i}$. | **Regularisation**: thêm số hạng phạt vào cost. (a) tổng bình phương trọng số = **L2 / ridge regression**, sklearn đặt bằng 'alpha' nhân tổng bình phương trọng số; (b) tổng \|trọng số\| = **L1 / lasso regression**. |
| **Non-linear regression bằng mạng nơ-ron** | Tuyến tính: ánh xạ trực tiếp input → output bằng trọng số. Phi tuyến: trước hết ánh xạ input → lớp ẩn (bằng bất kỳ gì, như trọng số/nơ-ron), rồi ánh xạ lớp ẩn → output tuyến tính. | **alpha** để regularisation: phạt trọng số lớn, tránh overfitting. Cost = tổng bình phương sai số + alpha × tổng bình phương trọng số. |
| **Gaussian Process Regression (GPR)** | Cách tiếp cận **Bayesian**. Mô hình GP là một phân phối xác suất trên các hàm có thể khớp một tập điểm. P(f\|X) = N(f\|μ, K), μ là hàm trung bình, K là hàm kernel cho hiệp phương sai. Cho ra phân phối dự đoán trên các hàm, hình dạng do K quyết định. | **Cách nghĩ rất khác**: ít tham số cần học (ví dụ 3), có **error bar** cho dự đoán, dự đoán phụ thuộc **trực tiếp vào dữ liệu trước đó**, nhưng **mất hiệu quả ở không gian nhiều chiều**. So với mạng lớn: hàng triệu tham số, không có error bar, dự đoán phụ thuộc tham số chứ không trực tiếp dữ liệu quá khứ. |

## 3C. Giảm chiều (DL04c)

- Dữ liệu có thể nằm trên một **manifold** (không gian con), ví dụ dữ liệu nằm trên một 'mặt phẳng' 2D trong không gian đặc trưng 3D. Kỹ thuật giảm chiều ánh xạ mỗi điểm vào không gian chiều thấp.
- Mục đích: (1) **insight**: nhân tố 'thật' bên dưới; (2) **tiền xử lý**: đưa phiên bản chiều thấp cho bộ phân loại để tránh overfitting.

| Kỹ thuật | Ý tưởng |
|---|---|
| **PCA** | Nếu manifold là tuyến tính (một 'mặt phẳng'), dễ tìm. PCA 'xoay không gian' $(x_0,x_1,x_2) \to (z_0,z_1,z_2)$: $z_0$ là hướng biến thiên nhiều nhất (**thành phần chính thứ nhất**), $z_1$ là hướng nhiều thứ hai, v.v. Nếu $z_2$ ít biến thiên thì có thể **bỏ**: $(x_0,x_1,x_2) \to (z_0,z_1)$. Dùng phép biến đổi **trực giao** để chuyển các biến có thể tương quan thành các biến **không tương quan (tuyến tính)**. Thành phần chính là tổ hợp tuyến tính của biến gốc. |
| **ICA** | Giả định mỗi biến dữ liệu là hỗn hợp của các thành phần **độc lập**, tìm các thành phần độc lập đó; đầu ra là **ma trận nguồn** với các cột là các nguồn độc lập. |
| **Autoencoder** | Loại mạng nơ-ron đặc biệt; huấn luyện để **sao chép đầu vào ra đầu ra**; có một tầng ẩn 'mỏng' mô tả **mã (code)** biểu diễn đầu vào; cực tiểu hóa **reconstruction loss**. |

## 3D. Phân cụm (DL04c)

**Clustering**: nhóm một tập đối tượng sao cho các đối tượng trong cùng nhóm (cluster) giống nhau hơn so với các đối tượng ở nhóm khác.

| Thuật toán | Ý tưởng | Tham số/điểm cần nhớ |
|---|---|---|
| **K-Means** | Rất đơn giản nhưng là ý tưởng sâu hơn. Bắt đầu với centroid ngẫu nhiên, lặp hai bước: **reassign** (mỗi điểm 'thuộc' về ai) và **updating** (tính trung bình các cụm mới, dời centroid). | Câu hỏi: (1) có đang cực tiểu hóa gì không? Có: 'score' là tổng bình phương khoảng cách tới centroid gần nhất; (2) k đúng là bao nhiêu? (hình **elbow method**: chỗ 'khuỷu' gợi ý k = 5 trong ví dụ); (3) đo khoảng cách: đơn giản nhất là Euclid; (4) centroid: đơn giản nhất là trọng tâm (center of mass); (5) khi nào dừng: hội tụ hoặc số vòng lặp tối đa. |
| **Mini-Batch K-Means** | Biến thể dùng mini-batch (tập con dữ liệu lấy ngẫu nhiên mỗi lần lặp) để giảm thời gian tính. | Thường chỉ kém K-Means thường một chút. |
| **Mean Shift** | Thuật toán cửa sổ trượt tìm vùng dày đặc điểm dữ liệu; mục tiêu là tìm tâm của mỗi nhóm; ứng viên centroid là trung bình các điểm trong cửa sổ; loại các cửa sổ gần trùng nhau. | Cần định nghĩa **bandwidth**, **không cần số cụm**. |
| **Gaussian Mixture Models (GMM)** | Khớp một hỗn hợp gồm K Gaussian (K là số cụm). Tham số: trung bình μ, hiệp phương sai σ, xác suất trộn π. Tìm bằng **Expectation–Maximization**: bắt đầu với Gaussian ngẫu nhiên; mỗi Gaussian 'sở hữu' mỗi điểm bao nhiêu?; dời cả trung bình và hiệp phương sai để sinh dữ liệu tốt hơn; lặp. | Là 'K-means mềm' (soft) và là **mô hình sinh** của dữ liệu. Có thể ràng buộc hiệp phương sai: **spherical, diagonal, tied hoặc full**. Hình Iris: độ chính xác train/test lần lượt spherical 88,4/92,1; diag 89,3/94,7; tied 95,5/100,0; full 87,5/89,5. |
| **Affinity Propagation** | Chọn các **exemplar** trong các điểm dữ liệu và tạo cụm quanh chúng. Dựa trên 'truyền thông điệp' (message passing): các thông điệp giá trị thực được trao đổi đệ quy giữa các điểm cho tới khi xuất hiện tập exemplar và cụm chất lượng cao. Dùng belief propagation để tìm kiếm và chấm điểm. | Chất lượng do **affinity** giữa các điểm quyết định; với dữ liệu thực, lựa chọn phổ biến là âm khoảng cách Euclid. Mọi thứ đều là điểm dữ liệu, **không có centroid riêng**. Lặp hai bước. |
| **DBSCAN** | Density-based spatial clustering of applications with noise. Coi cụm là vùng **mật độ cao** ngăn cách bởi vùng mật độ thấp. Ý chính: một điểm thuộc cụm nếu nó gần nhiều điểm của cụm đó. | Hai tham số: **eps** (khoảng cách xác định vùng lân cận), **minPts** (số điểm tối thiểu để định nghĩa cụm). Điểm là **core samples** (vùng mật độ cao) hoặc **outlier**. Cụm = tập core samples dựng đệ quy: lấy một core sample, tìm mọi láng giềng là core sample, rồi tiếp tục. |
| **Hierarchical, Agglomerative** | Họ thuật toán xây cụm lồng nhau bằng cách gộp hoặc tách; thể hiện bằng **cây/dendrogram**. Agglomerative là **bottom-up**: mỗi quan sát khởi đầu là một cụm, rồi các cụm được gộp dần. | **Linkage criteria** quyết định thước đo dùng cho chiến lược gộp. Cắt dendrogram ở các độ cao khác nhau cho k = 2, 3, 6... Slide nhấn: có nhiều lựa chọn để quyết định hai nhóm nào gộp tiếp. |

| Thuật toán | Ý tưởng | Tham số/điểm cần nhớ |
|---|---|---|
| **BIRCH** | Balanced Iterative Reducing Clusters using Hierarchies. Phân cụm phân cấp: xây cây **Clustering Feature Tree (CFT)** cho dữ liệu, dùng CF để tóm tắt một cụm. | Hay dùng để **bổ trợ** các thuật toán khác khi phân cụm tập dữ liệu lớn: tạo bản tóm tắt tập dữ liệu để thuật toán khác dùng. |

### PHÂN TÍCH CỦA MÌNH (không có trong slide)

- **So sánh nhanh (mình tổng hợp từ bảng trên):** cần chỉ định số cụm: K-Means, Mini-Batch, GMM; không cần chỉ định số cụm mà cần tham số khác: Mean Shift (bandwidth), DBSCAN (eps, minPts); phân cấp cho nhiều k cùng lúc: hierarchical (chọn k bằng cách cắt dendrogram); xử lý dữ liệu lớn: Mini-Batch, BIRCH; gán mềm (soft): GMM; nhận diện outlier: DBSCAN (outlier là điểm không thuộc vùng đậm đặc).
- Hình GMM chỉ là một bộ dữ liệu (Iris) với một lần chạy, nên không kết luận được loại hiệp phương sai nào 'tốt nhất' nói chung. Chú ý 'tied' có test 100,0 còn 'full' chỉ 89,5 trong hình: loại phức tạp nhất không nhất thiết tốt nhất (liên hệ overfitting ở bước 4).

### TỰ KIỂM TRA CỦA BƯỚC NÀY

- **Đối chiếu nguồn:** DL04b trang 47–64 (đã xem chữ; hình kNN, SVM, DT chỉ có chú thích). DL04c trang 4–38 (đã xem ảnh trang 13, 23, 28, 34: PCA, K-Means elbow, GMM covariance, dendrogram).
- **Ý tác giả và của mình:** toàn bộ mô tả từng họ lấy từ slide. Bảng 'so sánh nhanh' và nhận xét về hình GMM là của mình (đã gắn nhãn).
- **Lỗi/lệch trong slide:** cách gọi 'Perceptron = Logistic Regression' (xem hộp ⚠). Slide DL04c: 'DBSCN', 'centeriod', 'quantify output', 'Lectore 06'; DL04c trang 2 và 18 liệt kê hai danh sách clustering hơi khác nhau (trang 2 không có Mean Shift, BIRCH).
- **Chắc chắn:** cao cho mô tả. Chưa xác minh: các con số train/test của GMM đọc từ ảnh nhỏ; tham số cụ thể của từng thuật toán trong sklearn ngoài các tên slide đã nêu (C, alpha, var_smoothing, max_depth).

BƯỚC 4

# Kiểm soát độ phức tạp

Bias–variance · underfitting/overfitting · sai số tổng quát hóa · Occam

## 4.1. Bias và variance (DL04a)

**KEY POINT**

- Hình: hai tập dữ liệu mẫu từ **cùng một hàm f** (điểm) cùng hàm h đã học (đường), với bốn lớp mô hình: **Linear, Sinusoidal, Piecewise linear, Degree-12 polynomial**.
- **Bias**: giới hạn do lớp mô hình áp đặt. **Variance**: mức khác nhau của mô hình khi dữ liệu thay đổi chút ít.
- Từ trái sang phải: bias **cao → thấp**, variance **thấp → cao**; hai trục 'simplicity (ít tham số)' và 'consistency (khớp dữ liệu huấn luyện)'. Các mô hình phức tạp nhất bị **overfitting** vào dữ liệu huấn luyện.
- Ghi chú: **sự đánh đổi này áp dụng cho mọi mô hình**.

## 4.2. Underfitting và overfitting

| Hiện tượng | Mô tả | Dấu hiệu |
|---|---|---|
| **Underfitting** | Mô hình quá đơn giản | Cả sai số huấn luyện và sai số test đều lớn. |
| **Overfitting** | Mô hình quá phức tạp, bắt đầu 'học thuộc' dữ liệu huấn luyện | Sai số huấn luyện tiếp tục giảm nhưng **sai số tổng quát hóa tăng trở lại**. |

- **Ví dụ của slide:** 500 điểm tròn và 500 điểm tam giác trong $[-1,1]^2$. Điểm tròn thuộc vành $0,5 \le \sqrt{(x_1^2 + x_2^2)} \le 1$. Điểm tam giác: $\sqrt{(\cdot)} < 0,5$ hoặc $\sqrt{(\cdot)} > 1$.
- **Đồ thị** (hình, trục x là số nút của cây, trục y là sai số %): sai số tập train (resubstitution) giảm đều từ khoảng 38% xuống khoảng 9%; sai số tập test (generalization) giảm tới khoảng 32% rồi tăng lại về khoảng 38%. Vùng giữa là **optimal complexity**. (Số đọc từ hình, chỉ ước lượng.)
- **Underfitting do thiếu ví dụ:** thiếu điểm huấn luyện ở nửa dưới của biểu đồ làm khó dự đoán đúng nhãn ở vùng đó.
- **Overfitting do nhiễu:** ranh giới quyết định bị bóp méo để chứa một điểm nhiễu.

## 4.3. Sai số huấn luyện và sai số tổng quát hóa

- Sai số huấn luyện bị giảm nhờ overfitting, dẫn đến cây phức tạp hơn mức cần thiết; **sai số huấn luyện không là ước lượng tốt** cho hiệu năng trên mẫu mới (ví dụ dữ liệu test). Cần ước lượng **sai số tổng quát hóa** kỳ vọng cho dữ liệu mới.
- **Resubstitution error e**: sai số trên tập huấn luyện. **Generalization error e'**: sai số trên tập kiểm thử (có thể thêm 'penalty cho độ phức tạp mô hình').

| Cách ước lượng | Nội dung |
|---|---|
| 1. **Optimistic** | Giả định e' = e. |
| 2. **Pessimistic** | e' = e + N × 0,5 (N là số nút lá); 0,5 mỗi nút lá thường dùng cho chia nhị phân. Ví dụ: cây 30 nút lá, 10 lỗi trên 1000 mẫu huấn luyện: e = 10/1000 = 1%; e' = (10 + 30 × 0,5)/1000 = **2,5%** (đã kiểm). |
| 3. **Validation** | Dùng tập validation (test) hoặc cross-validation để ước lượng sai số tổng quát hóa. |

## 4.4. Occam's razor (Principle of Parsimony)

**KEY POINT**

- Cho hai mô hình có sai số tổng quát hóa tương đương, **nên chọn mô hình đơn giản hơn**. Lý do: mô hình phức tạp có nhiều khả năng overfitting hơn (khớp nhầm cả lỗi ngẫu nhiên trong dữ liệu huấn luyện). Vì vậy khi đánh giá mô hình, **phải xét cả độ phức tạp**.

## 4.5. Tuning một phương pháp so với đánh giá nó (DL04b)

- Ví dụ: nên dùng k bằng bao nhiêu trong kNN? **k = 1 sẽ tốt nhất trên chính dữ liệu đã huấn luyện**, nhưng đó không phải điều ta cần: nó chạy thế nào trên dữ liệu khác?
- Cần ước lượng 'giá trị' mô hình (ví dụ % đúng): chia dữ liệu thành tập Training và tập 'giữ lại' (Test).
- Câu hỏi của slide: 'cứ dùng mô hình có accuracy validation cao nhất?' Trả lời: **không!** Chia thành Training+Validation và Test.

**PHÂN TÍCH CỦA MÌNH (không có trong slide)**

- Slide không giải thích vì sao 'không'. Lý do mình suy ra: nếu chọn mô hình có điểm validation cao nhất trong nhiều ứng viên, thì điểm validation đó đã bị 'chọn lọc' nên lạc quan hơn thực tế; cần tập test chưa từng dùng để ước lượng trung thực. Khớp với câu trong DL04a: 'We do not want to overfit the hyperparameters!!!'

### Bảng tổng hợp: nút vặn độ phức tạp của từng mô hình

| Mô hình | Tham số | Giá trị thấp / cao |
|---|---|---|
| kNN | k | Thấp: bám láng giềng gần nhất, dễ overfit. Cao: trung bình nhiều láng giềng, dễ underfit. |
| Logistic Regression | C (sklearn) | Nghịch đảo độ mạnh regularization: C nhỏ → regularization mạnh. |
| MLP / NN regression | alpha | Độ mạnh số hạng phạt L2; phạt trọng số lớn → ranh giới trơn hơn. |
| Naïve Bayes | Laplace smoothing / var_smoothing | Tránh overfit khi đếm bằng 0 hoặc đếm rất thấp; đếm >> 1 thì ảnh hưởng ít. |
| Decision Tree, Boosting, Random Forest | max_depth | Độ sâu cây càng lớn càng phức tạp. |
| Ridge/Lasso | alpha (sklearn) | Hệ số phạt tổng bình phương (L2) hoặc tổng trị tuyệt đối (L1) của trọng số. |
| K-Means/GMM | k; kiểu hiệp phương sai | Số cụm; spherical, diagonal, tied, full. |

**TỰ KIỂM TRA CỦA BƯỚC NÀY**

- **Đối chiếu nguồn:** DL04a trang 6–13 (đã xem ảnh trang 6, 7, 8); DL04b trang 51–52. Công thức pessimistic đã kiểm bằng tay (2,5%).
- **Ý tác giả và của mình:** định nghĩa, ví dụ, công thức là của slide. Lý giải 'vì sao không chọn validation cao nhất' là của mình; bảng tổng hợp nút vặn là mình gom từ các slide khác nhau.
- **Lỗi trong slide:** trang 7 ghi vành điểm tròn là '0,5 ≥ √(...) ≤ 1' (đã xem ảnh); đúng phải là 0,5 ≤ √(...) ≤ 1, như bản trên đây. Cũng ở DL04a, mục 'Topics' ghi 'Decision Trees (Overview, Tree Induction)' nhưng file này **không có** slide về cây quyết định, trong khi trang 11–12 giả định người học đã biết cây quyết định.
- **Chắc chắn:** cao cho định nghĩa. Trung bình cho các số đọc từ đồ thị (38%, 9%, 32%) và nội dung hình bias–variance.

BƯỚC 5

# Thước đo hiệu năng

Nguồn: DL04a (p15–26)

Các thước đo tập trung vào **khả năng dự đoán** của mô hình (không xét tốc độ, khả năng mở rộng). Slide trình bày cho phân loại nhị phân, nhưng hầu hết thước đo tổng quát hóa được cho đa lớp.

## 5.1. Confusion matrix

| | Dự đoán = Yes | Dự đoán = No |
|---|---|---|
| Thực tế = Yes | a: **TP** (true positive) | b: **FN** (false negative) |
| Thực tế = No | c: **FP** (false positive) | d: **TN** (true negative) |

- Góc nhìn kiểm định thống kê: giả thuyết không H0 là 'lớp thực tế là Yes'. **Type I error** = FN, P(No | H0 đúng) → mức ý nghĩa α. **Type II error** = FP, P(Yes | H0 sai) → liên quan tới power 1 − β.

## 5.2. Accuracy và giới hạn của nó

- **Accuracy** = (a + d)/(a + b + c + d) = (TP + TN)/N. Thước đo dùng rộng rãi nhất: dự đoán đúng bao nhiêu phần trăm.
- **Class imbalance:** bài toán 2 lớp, 9990 mẫu lớp 0 và 10 mẫu lớp 1. Mô hình luôn dự đoán lớp 0 có accuracy 9990/10000 = **99,9%**, nhưng **không phát hiện mẫu lớp 1 nào**: accuracy gây hiểu lầm. Đây là vấn đề rất phổ biến gọi là **class imbalance problem**.

**PHÂN TÍCH CỦA MÌNH (không có trong slide)**

- Với mô hình luôn dự đoán lớp 0 và coi lớp 1 là 'positive': recall = 0/10 = 0 (không bắt được mẫu positive nào) và precision = 0/0 (không xác định, vì không có dự đoán Yes). Slide không tính; đây là mình áp dụng định nghĩa ở mục 5.4.

## 5.3. Ma trận chi phí (cost matrix)

- Các loại lỗi có thể có **chi phí khác nhau**. C(i|j) là chi phí khi phân loại nhầm mẫu lớp j thành lớp i.
- Ví dụ slide: C(+|+) = −1, C(−|+) = 100 ('bỏ sót ca +' rất đắt), C(+|−) = 1, C(−|−) = 0.

| | TP | FN | FP | TN | Accuracy | Cost |
|---|---|---|---|---|---|---|
| Model M1 | 150 | 40 | 60 | 250 | 80% | −150 + 100×40 + 1×60 + 0×250 = **3910** |
| Model M2 | 250 | 45 | 5 | 200 | 90% | −250 + 100×45 + 1×5 + 0×200 = **4255** |

**PHÂN TÍCH CỦA MÌNH (không có trong slide)**

- Bài học: M2 **chính xác hơn** (90% so với 80%) nhưng **tốn kém hơn** (4255 so với 3910), vì chi phí chủ yếu đến từ FN (100 mỗi lỗi) và M2 có FN nhiều hơn (45 so với 40). Chọn mô hình theo accuracy hay theo cost có thể cho kết quả khác nhau (đã kiểm các phép tính, khớp slide).

## 5.4. Precision, Recall, F-measure (thước đo thiên về chi phí)

| Thước đo | Công thức | Ý nghĩa |
|---|---|---|
| **Precision** p | a / (a + c) = TP / (TP + FP) | Chỉ xét chi phí cho các mẫu **được dự đoán là Yes**. |
| **Recall** r | a / (a + b) = TP / (TP + FN) | Chỉ xét chi phí cho các mẫu **thực sự là Yes**. |
| **F-measure** | F = 2rp / (r + p) = 2a / (2a + b + c) | Kết hợp precision và recall; **bỏ qua d** (TN). |

## 5.5. Kappa statistic

- Ý tưởng: so sánh accuracy của bộ phân loại với một **bộ phân loại ngẫu nhiên**; bộ phân loại nên tốt hơn ngẫu nhiên.
- κ = (total accuracy − random accuracy) / (1 − random accuracy); total accuracy = (TP + TN)/N.

**⚠ LỖI / ĐIỂM LỆCH TRONG SLIDE**

- Công thức **random accuracy** trên slide (đã phóng to ảnh) viết 'TP + FP × TN + FN + FN + TN × FP + TP' trên $N^2$, không có dấu ngoặc. Nếu hiểu các cặp là (TP+FP)(TN+FN) và (FN+TN)(FP+TP) thì **hai tích giống hệt nhau**, và cũng không phải công thức của Cohen.
- Công thức chuẩn mình dùng: random accuracy = [(TP+FN)(TP+FP) + (FP+TN)(FN+TN)] / $N^2$ (tức tổng tích của tổng hàng và tổng cột tương ứng, chia $N^2$). Đây là kiến thức ngoài slide (mức chắc chắn cao).

### Áp dụng cho M1 và M2 (mình tính, đã kiểm bằng code)

| | Precision | Recall | F | Random accuracy | Kappa |
|---|---|---|---|---|---|
| M1 | 150/210 = 0,714 | 150/190 = 0,789 | 300/400 = 0,750 | 0,519 | 0,584 |
| M2 | 250/255 = 0,980 | 250/295 = 0,847 | 500/550 = 0,909 | 0,502 | 0,799 |

**PHÂN TÍCH CỦA MÌNH (không có trong slide)**

- Slide không tính các số này. Bảng chỉ để luyện công thức. Lưu ý M2 vượt trội trên mọi thước đo này trong khi lại tốn kém hơn về cost: cho thấy thước đo phải khớp mục tiêu thực tế.

## 5.6. ROC và AUC

- **ROC** (Receiver Operating Characteristic): phát triển từ thập niên 1950 trong lý thuyết phát hiện tín hiệu, để mô tả sự đánh đổi giữa 'hits' và 'false alarms'. Chỉ dùng cho **phân loại nhị phân**.
- Đường ROC vẽ **TPR** (trục y) theo **FPR** (trục x). Hiệu năng mỗi bộ phân loại là một điểm; đổi ngưỡng của thuật toán, phân phối mẫu hoặc ma trận chi phí thì điểm dịch chuyển và tạo thành đường cong.
- Ví dụ 1 chiều: điểm có x > t được phân loại là positive. Tại ngưỡng t: TPR = 0,5, FNR = 0,5, FPR = 0,12, TNR = 0,88. Dịch t để có các điểm khác.
- Điểm đặc biệt (TPR, FPR): **(0,0)**: khai báo mọi thứ là negative; **(1,1)**: mọi thứ là positive; **(1,0)**: lý tưởng. **Đường chéo**: đoán ngẫu nhiên; dưới đường chéo: dự đoán ngược với lớp thật (nên đảo dự đoán).
- **So sánh mô hình bằng ROC:** không mô hình nào luôn vượt trội: M1 tốt hơn khi FPR nhỏ, M2 tốt hơn khi FPR lớn. **AUC** (diện tích dưới đường ROC): lý tưởng AUC = 1; ngẫu nhiên AUC = 0,5.

**⚠ LỖI / ĐIỂM LỆCH TRONG SLIDE**

- Slide ghi 'TPR=0.5, FNR=0.5, FPR=0.12, FNR=0.88': chữ FNR thứ hai là lỗi, phải là **TNR** (vì FPR + TNR = 0,12 + 0,88 = 1).

**TỰ KIỂM TRA CỦA BƯỚC NÀY**

- **Đối chiếu nguồn:** DL04a trang 15–26; đã xem ảnh trang 16, 20, 22, 25, 26. Mọi tính toán (cost M1, M2; precision/recall/F/kappa) kiểm bằng code; 99,9% tính tay.
- **Ý tác giả và của mình:** định nghĩa và công thức (trừ random accuracy) là của slide. Bảng P/R/F/κ cho M1, M2, nhận xét về cost và công thức Cohen chuẩn là của mình (gắn nhãn).
- **Lỗi trong slide:** công thức random accuracy; 'FNR=0.88' (TNR); mô tả Type II error có chữ 'Power 1 − β' đặt cạnh Type II dễ gây hiểu rằng power là xác suất lỗi loại II (thực ra β là xác suất lỗi loại II, power = 1 − β).
- **Chắc chắn:** cao cho các định nghĩa chính; trung bình cho cách hiểu công thức random accuracy (không có dấu ngoặc trong slide).
- **Giới hạn:** không có thước đo cho regression hay clustering trong ba file (DL04c chỉ nhắc 'metrics and scoring').

# BƯỚC 6 Ước lượng đáng tin cậy và so sánh mô hình

Nguồn: DL04a (p28–37), DL04b (p52), DL04c (p38)

## 6.1. Learning curve

- Accuracy và độ biến thiên giữa các lần chạy phụ thuộc vào **kích thước dữ liệu huấn luyện**.
- Learning curve cho thấy accuracy trên mẫu chưa thấy thay đổi thế nào khi tăng cỡ mẫu huấn luyện. Hình (trục x log): accuracy tăng từ khoảng 61% (rất ít mẫu) lên khoảng 90% (hàng nghìn mẫu), thanh sai số giữa các lần chạy **thu hẹp dần**. (Số đọc từ hình.)

## 6.2. Ước lượng sai số tổng quát hóa bằng dữ liệu test

| Phương pháp | Cách làm | Điểm cần nhớ |
|---|---|---|
| **Holdout / random split** | Chia ngẫu nhiên, ví dụ 80% train và 20% test. | **Rất quan trọng: thuật toán không bao giờ được nhìn tập test trong lúc học.** |
| **k-fold cross validation** | Chia ngẫu nhiên thành k fold. Qua k vòng, mỗi vòng giữ lại 1 fold để test và dùng k−1 fold còn lại để train. Dùng **trung bình** lỗi/accuracy làm ước lượng tốt hơn. | Dùng dữ liệu tốt hơn. Một số thuật toán/công cụ tự làm việc này bên trong. (Hình: shuffle, 10 fold, lỗi trên fold 1..10, lấy trung bình.) |

## 6.3. Huấn luyện và kiểm thử với siêu tham số

**KEY POINT**

- **Siêu tham số (hyperparameters):** nhiều thuật toán cho phép chọn khi học, ví dụ **độ sâu tối đa của cây**, **các đặc trưng được chọn**.
- Ta không muốn **overfit siêu tham số**. Dùng ước lượng sai số tổng quát hóa **hai lần**.

| Bước | Việc làm |
|---|---|
| 1. **Train** | Học các mô hình trên dữ liệu huấn luyện (không gồm validation) với các siêu tham số khác nhau: một **lưới (grid)** các tổ hợp siêu tham số, hoặc **tìm kiếm tham lam (greedy)**. |
| 2. **Model selection** | Đánh giá các mô hình bằng **validation data** và chọn siêu tham số có accuracy tốt nhất. **Dựng lại mô hình bằng toàn bộ dữ liệu huấn luyện.** |
| 3. **Test** | Kiểm tra mô hình cuối cùng bằng **test data**. |

- **Cách dùng dữ liệu điển hình (hình):** tách ngẫu nhiên **20% test** và **80% training + validation**. Model selection: dùng training và validation với **10-fold cross validation** để chọn giữa các mô hình và tinh chỉnh siêu tham số.

## 6.4. Khoảng tin cậy cho accuracy

- Accuracy quan sát được chỉ là **ước lượng** accuracy thật. Ước lượng tốt đến đâu?
- Mỗi dự đoán là một **phép thử Bernoulli** (đồng xu thiên lệch): đúng hoặc sai. p là xác suất thật dự đoán đúng (= accuracy thật). Với tập test cỡ N, số dự đoán đúng X ~ **Binomial(N, p)**; E[X] = Np (ví dụ tung đồng xu cân bằng 50 lần: kỳ vọng 50 × 0,5 = 25 mặt ngửa). Accuracy quan sát $\hat{p} = x/N$.
- Với tập test lớn (**N > 30**), xấp xỉ Binomial bằng **Normal**: X ~ Normal(Np, Np(1−p)). **Khoảng tin cậy Wald**: $\hat{p} \pm z_{\alpha/2} \cdot \sqrt{\hat{p}(1-\hat{p})/N}$, với độ tin cậy $1 - \alpha$.

### Ví dụ của slide (đã kiểm): accuracy 80% trên N = 100, tin cậy 95%

| Bước | Kết quả |
|---|---|
| N = 100, $\hat{p}$ = 0,8 | — |
| $1 - \alpha = 0{,}95 \rightarrow z_{\alpha/2}$ | 1,96 |
| $\hat{p} \pm z \cdot \sqrt{\hat{p}(1-\hat{p})/N} = 0{,}8 \pm 1{,}96 \times \sqrt{0{,}16/100}$ | 0,8 ± 0,0784 → **[0,722; 0,878]** |

| Độ tin cậy | $z_{\alpha/2}$ (bảng slide) | Kiểm bằng qnorm |
|---|---|---|
| 0,99 | 2,58 | 2,576 |
| 0,98 | 2,33 | 2,326 |
| 0,95 | 1,96 | 1,960 |
| 0,90 | 1,65 | 1,645 |

**⚠ LỖI / ĐIỂM LỆCH TRONG SLIDE**

- Bảng z trên slide có cột đầu ghi '1 − α/2' nhưng các giá trị (0,99; 0,98; 0,95; 0,90) thực ra là **độ tin cậy 1 − α**: $z_{\alpha/2}$ = qnorm(1 − α/2), ví dụ độ tin cậy 95% ứng với qnorm(0,975) = 1,96. Đã kiểm bằng code. Slide cũng ghi 'Table or R qnorm(1 − α/2)'.

**PHÂN TÍCH CỦA MÌNH (không có trong slide)**

- Khoảng tin cậy **hẹp dần khi N tăng**: với $\hat{p}$ = 0,8, N = 100 cho nửa độ rộng ±7,84%; N = 1000 cho ±2,48% (mình tính từ công thức Wald, slide không nêu). Vì vậy cùng accuracy 80%, kết quả trên tập test nhỏ kém đáng tin hơn.

## 6.5. So sánh hai mô hình và nhiều mô hình

- Cho hai mô hình M1 và M2, mô hình nào tốt hơn? Đây là bài toán **model selection mang tính thống kê**.
- Với tập test lớn (N > 30), xấp xỉ accuracy quan sát bằng phân phối Normal với $p_1$, $p_2$ thật nhưng chưa biết. Thực hiện **paired t-test**: H0: **không có khác biệt** giữa accuracy quan sát của hai mô hình; H1: có khác biệt.
- Tinh chỉnh siêu tham số cũng là một bài toán model selection.
- So sánh **hơn hai mô hình**: phải **hiệu chỉnh cho so sánh bội (multiple comparisons)**, ví dụ **Bonferroni correction** hoặc **False Discovery Rate (FDR)**.
- DL04c: model selection gồm cross-validation, tinh chỉnh (hyper)parameters của learner, metrics và scoring để chọn giữa các mô hình (tham chiếu scikit-learn model_selection).

**PHÂN TÍCH CỦA MÌNH (không có trong slide)**

- Giải thích ngoài slide: so sánh nhiều lần thì xác suất 'tình cờ thấy khác biệt' tăng lên, nên cần siết ngưỡng; Bonferroni làm điều đó bằng cách chia mức ý nghĩa α cho số lần so sánh (kiến thức ngoài slide, mức chắc chắn cao).

**⚠ LƯU Ý VỀ KÝ HIỆU**

- Slide ghi '$acc_1$ ~ Normal($Np_1$, $Np_1(1-p_1)$)': các tham số này (kỳ vọng Np, phương sai Np(1−p)) là của **số dự đoán đúng**, không phải của tỉ lệ accuracy. Hiểu 'acc' ở đây là số lần đúng.

**TỰ KIỂM TRA CỦA BƯỚC NÀY**

- **Đối chiếu nguồn:** DL04a trang 28–37 (đã xem ảnh trang 28, 31, 32, 35); DL04b trang 52; DL04c trang 38. Kiểm bằng code: khoảng tin cậy [0,722; 0,878] và bảng z (qnorm).
- **Ý tác giả và của mình:** quy trình, công thức, ví dụ là của slide. Giải thích Bonferroni, ví dụ N = 1000 là của mình (gắn nhãn).
- **Lỗi trong slide:** tiêu đề cột bảng z (1−α/2 thay vì 1−α); ký hiệu acc ở trang 37.
- **Chắc chắn:** cao. Chưa xác minh: slide không nêu công thức thống kê của paired t-test và cách hiệu chỉnh Bonferroni/FDR.

# BƯỚC 7 Chọn đặc trưng và quy trình tổng

Nguồn: DL04a (p39–40), DL04b (p41), DL04c (p37–38)

## 7.1. Feature selection (DL04a)

**KEY POINT**

- Câu hỏi: **nên dùng những đặc trưng nào trong mô hình?**

| | Univariate feature importance score | Feature subset selection |
|---|---|---|
| Ý tưởng | Đo **mỗi đặc trưng liên quan thế nào** với biến lớp. | Cố gắng tìm **tập đặc trưng tốt nhất**. |
| Cách làm | Ví dụ thống kê chi-squared, information gain. | Thường dùng cách tiếp cận **black box**: các tập con được đánh giá bằng chiến lược tìm kiếm **tham lam**. |
| Ví dụ | chi-squared, information gain | **Stepwise backward selection**: thử loại một đặc trưng mỗi lần. |

**PHÂN TÍCH CỦA MÌNH (không có trong slide)**

- Nội dung chi tiết về chọn đặc trưng (filter, wrapper, embedded, SFG/SBG, PCA...) nằm ở bộ file DL03 (đã soạn trong tài liệu ôn trước). DL04a chỉ nêu tóm tắt. Mình chỉ nối quan hệ, không lặp lại.

## 7.2. Kết luận của DL04a

**KẾT LUẬN CỦA SLIDE**

- Classification là học có giám sát với mục tiêu tìm mô hình **dự đoán tốt** (tức có **sai số tổng quát hóa thấp**).
- Sai số tổng quát hóa được ước lượng bằng tập test/cross-validation và **nên dùng để chọn mô hình**.
- Đánh giá và so sánh mô hình cần **tính đến độ phức tạp mô hình**.

## 7.3. Quy trình tham chiếu để chọn và đánh giá mô hình (checklist)

| # | Việc cần làm | Cơ sở (nguồn) |
|---|---|---|
| 1 | Xác định nhiệm vụ: classification, regression, clustering, giảm chiều (và dữ liệu có nhãn hay không) | DL04b p27, DL04c p20 |
| 2 | Biểu diễn dữ liệu như vector; lưu ý số chiều (curse of dimensionality) | DL04b p11–15 |
| 3 | Chọn một vài họ mô hình ứng viên phù hợp nhiệm vụ | DL04b p47–64, DL04c |
| 4 | Tách test (ví dụ 20%) và **không nhìn** tới khi kết thúc | DL04a p29, p32 |
| 5 | Trên phần còn lại (80%): dùng 10-fold CV để chọn mô hình và siêu tham số (grid/greedy) | DL04a p30–32 |
| 6 | Cân nhắc độ phức tạp: bias–variance, Occam, regularisation (k, C, alpha, max_depth...) | DL04a p6–13, DL04b p51–55 |
| 7 | Dựng lại mô hình tốt nhất bằng toàn bộ phần train+validation | DL04a p31 |
| 8 | Đánh giá **một lần** trên test với thước đo phù hợp (accuracy, cost, P/R/F, kappa, ROC/AUC) | DL04a p15–26 |
| 9 | Báo cáo kèm khoảng tin cậy; so sánh mô hình bằng kiểm định (paired t-test) và hiệu chỉnh nếu so sánh nhiều lần | DL04a p33–37 |

**PHÂN TÍCH CỦA MÌNH (không có trong slide)**

- Bảng này là **tổng hợp của mình** từ các slide; thứ tự các bước không có trong một slide duy nhất. Cũng như ở tài liệu trước, mọi phép biến đổi dữ liệu 'học' từ dữ liệu (chuẩn hóa, điền thiếu, chọn đặc trưng) nên được fit chỉ trên tập train để không rò rỉ (nguyên tắc leakage từ DL03b).

## Ví dụ code minh họa (đã chạy thử)

**PHÂN TÍCH CỦA MÌNH (không có trong slide)**

- Slide **không có** đoạn code này. Mình viết để minh họa quy trình ở trên, trên bộ dữ liệu breast cancer có sẵn trong sklearn (sklearn 1.8.0). Kết quả lần chạy: k tốt nhất = 3 (CV accuracy ≈ 0,969); test accuracy ≈ 0,956, khoảng tin cậy 95% ≈ [0,919; 0,994]; precision/recall/F1 ≈ 0,947/0,986/0,966. Số liệu chỉ để xác nhận code chạy, không phải kết quả thực nghiệm của bài tập.

```
import math
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, precision_score, recall_score, f1_score

X, y = load_breast_cancer(return_X_y=True)
# 1) Giữ lại 20% làm TEST, chưa đụng tới cho đ□n cu□i
X_dev, X_te, y_dev, y_te = train_test_split(X, y, test_size=0.2,
                                            stratify=y, random_state=0)
# 2) Chọn mô hình/siêu tham s□ bằng 10-fold CV trên ph□n 80%
pipe = Pipeline([("sc", StandardScaler()), ("knn", KNeighborsClassifier())])
grid = GridSearchCV(pipe, {"knn__n_neighbors": [1, 3, 5, 11, 25, 50]},
                    cv=10, scoring="accuracy")
grid.fit(X_dev, y_dev)              # refit=True: dựng lại trên toàn bộ 80%
# 3) Ki□m tra MỘT L□N trên test
y_hat = grid.predict(X_te)
acc = (y_hat == y_te).mean()
half = 1.96 * math.sqrt(acc * (1 - acc) / len(y_te))   # kho□ng tin cậy Wald 95%
print(grid.best_params_, grid.best_score_, acc, (acc - half, acc + half))
print(confusion_matrix(y_te, y_hat))
print(precision_score(y_te, y_hat), recall_score(y_te, y_hat), f1_score(y_te, y_hat))
```

**PHÂN TÍCH CỦA MÌNH (không có trong slide)**

- Phần *Pipeline* bảo đảm StandardScaler chỉ fit trên fold train trong mỗi vòng CV. Việc dùng Pipeline/GridSearchCV là thực hành chuẩn của sklearn, mình chưa đối chiếu với slide (slide chỉ nhắc tên model_selection).

**TỰ KIỂM TRA CỦA BƯỚC NÀY**

- **Đối chiếu nguồn:** DL04a trang 39–40, DL04b trang 41, DL04c trang 37–38. Code đã chạy thử trên dữ liệu có sẵn.
- **Ý tác giả và của mình:** 7.1 và 7.2 là của slide. Checklist 7.3 và code là của mình.
- **Giới hạn:** DL04a phần feature selection chỉ có một slide; DL04c phần 'What else? Preprocessing' chỉ là tiêu đề, không nội dung.
- **Chắc chắn:** cao cho 7.1, 7.2; trung bình cho checklist (suy luận từ nhiều slide).

# PHỤ LỤC

## A. Tổng hợp lỗi và điểm lệch trong slide

| # | File | Nội dung | Cách xử lý khi ôn |
|---|---|---|---|
| 1 | DL04a p7 | Vành điểm tròn ghi '0,5 ≥ √(...) ≤ 1' (đã xem ảnh) | Đúng: 0,5 ≤ √(...) ≤ 1 |
| 2 | DL04a p22 | Công thức random accuracy của kappa không có ngoặc, hai tích giống nhau nếu hiểu theo ngoặc tự nhiên | Dùng công thức Cohen chuẩn (xem bước 5.5) |
| 3 | DL04a p24 | 'FNR=0.12 ... FNR=0.88' (chữ FNR thứ hai) | Đọc là TNR |
| 4 | DL04a p35 | Cột đầu bảng z ghi '1−α/2' nhưng giá trị là độ tin cậy 1−α | Dùng $z_{\alpha/2}$ = qnorm(1−α/2): 90%→1,645; 95%→1,96; 98%→2,326; 99%→2,576 |
| 5 | DL04a p37 | $acc_1$ ~ Normal($Np_1$,...) là của số dự đoán đúng, không phải tỉ lệ | Hiểu acc là số lần đúng |
| 6 | DL04a p2, 14, 27, 36, 38 | Mục 'Decision Trees (Overview, Tree Induction)' không có slide trong file; p11–12 giả định đã học cây quyết định | Xem lại bài cây quyết định (không có trong ba file này) |
| 7 | DL04a p16 | 'Type II error → Power 1−β' dễ gây hiểu power là xác suất lỗi | β là xác suất lỗi loại II; power = 1 − β |
| 8 | DL04b p53 | 'Perceptron classifier (Logistic Regression)' coi hai thứ là một | Theo cách gọi của slide; hiểu: ngưỡng mềm = logistic regression |
| 9 | DL04c p2 và p18 | Hai danh sách clustering không giống nhau (p2 thiếu Mean Shift, BIRCH) | Dùng danh sách ở p18 (đầy đủ hơn) |
| 10 | DL04b, DL04c | 'Principle Components', 'Lectore 06', 'DBSCN', 'centeriod', 'quantify output' | Chỉ là lỗi chính tả |

## B. Thuật ngữ Anh – Việt

| Thuật ngữ | Nghĩa | Thuật ngữ | Nghĩa |
|---|---|---|---|
| supervised/unsupervised/reinforcement | có giám sát / không giám sát / học tăng cường | hyperparameter | siêu tham số |
| classification/regression | phân loại / hồi quy | regularisation (L1/L2) | điều chuẩn / phạt độ lớn trọng số |
| clustering | phân cụm | bias / variance | độ chệch / phương sai |
| dimension reduction | giảm chiều | underfitting / overfitting | chưa khớp / quá khớp |
| hypothesis space | không gian giả thuyết | generalization error | sai số tổng quát hóa |
| decision boundary | ranh giới quyết định | resubstitution error | sai số trên chính tập huấn luyện |
| confusion matrix | ma trận nhầm lẫn | cost matrix | ma trận chi phí |
| precision / recall | độ chính xác dương / độ phủ | ROC / AUC | đường cong ROC / diện tích dưới đường cong |
| class imbalance | mất cân bằng lớp | confidence interval | khoảng tin cậy |
| holdout / k-fold CV | giữ lại / kiểm chứng chéo k fold | multiple comparisons | so sánh bội |
| ensemble / bootstrap | tập hợp mô hình / mẫu lấy có hoàn lại | kernel trick | mẹo kernel |
| manifold | đa tạp | centroid / exemplar | tâm cụm / điểm đại diện |

## C. Câu hỏi tự kiểm tra (kèm đáp án gợi ý)

| # | Câu hỏi | Đáp án gợi ý |
|---|---|---|
| 1 | Phân biệt model và algorithm. Cho ví dụ. | Model: cơ sở lý thuyết của việc phân loại; algorithm: cách cụ thể thực hiện; phân biệt cách học và cách dùng mô hình (bước 1.1). |
| 2 | Bốn nhiệm vụ chính của ML và hai nhóm giám sát chứa chúng? | Classification, regression (supervised); clustering, dimension reduction (unsupervised) (bước 1.2–1.3). |
| 3 | Clustering khác classification ở ba điểm nào? | Số lớp (chưa biết/đã biết); dữ liệu huấn luyện (không bắt buộc/bắt buộc); mục đích (dữ liệu hiện có/phân loại mẫu tương lai) (bước 1.3). |
| 4 | Hai mục đích của classification? | Prediction và insight (bước 1.4). |
| 5 | Vì sao nói dữ liệu nhiều chiều 'ở các góc'? | Tỉ lệ điểm ngẫu nhiên trong quả cầu nội tiếp hộp giảm gần 0 khi d tăng (bước 2.3). |
| 6 | Để tóm tắt hàm bằng một giá trị mỗi 'góc phần tư' cần bao nhiêu số ở d chiều? | $2^d$ (suy luận của mình, bước 2.4). |
| 7 | Mô tả kNN và nút vặn độ phức tạp của nó. | Dự đoán theo láng giềng gần nhất; k thấp dễ overfit, k cao dễ underfit (bước 3A). |
| 8 | Naïve Bayes giả định gì? Hai vị của nó? | Y tạo mỗi $X_i$ độc lập; Multinomial (danh mục), Gaussian (số thực) (bước 3A). |
| 9 | Kernel trick là gì? | Ranh giới tuyến tính ở không gian chiều cao, phi tuyến ở không gian đầu vào; không dựng tường minh không gian đó (bước 3A). |
| 10 | GBDT khác Random Forest thế nào? | GBDT: cây dựng lần lượt, cộng dự đoán, cây sau bù phần chênh lệch; RF: mỗi cây từ bootstrap sample, tách nút từ tất cả hoặc tập con ngẫu nhiên đặc trưng (bước 3A). |
| 11 | Ridge và lasso khác nhau? | Ridge: phạt tổng bình phương trọng số (L2); lasso: phạt tổng trị tuyệt đối (L1) (bước 3B). |
| 12 | GPR có ưu và nhược gì so với mạng nơ-ron lớn? | Ít tham số, có error bar, dự đoán phụ thuộc trực tiếp dữ liệu; mất hiệu quả ở nhiều chiều (bước 3B). |
| 13 | PCA 'xoay không gian' nghĩa là gì? | Biến đổi trực giao, $z_0$ là hướng biến thiên nhiều nhất...; bỏ hướng ít biến thiên để giảm chiều (bước 3C). |
| 14 | K-Means tối thiểu hóa gì? Chọn k thế nào? | Tổng bình phương khoảng cách tới centroid gần nhất; elbow method (bước 3D). |
| 15 | DBSCAN có hai tham số nào, tạo ra loại điểm nào? | eps, minPts; core samples và outliers (bước 3D). |
| 16 | Mean Shift cần tham số gì mà không cần gì? | Cần bandwidth, không cần số cụm (bước 3D). |
| 17 | Giải thích bias–variance bằng bốn lớp mô hình trên hình. | Linear → degree-12: bias cao → thấp, variance thấp → cao; phức tạp nhất overfit (bước 4.1). |
| 18 | Tính sai số tổng quát hóa pessimistic: 30 lá, 10 lỗi, 1000 mẫu. | (10 + 30 × 0,5)/1000 = 2,5% (bước 4.3). |
| 19 | Phát biểu Occam's razor. | Hai mô hình sai số tổng quát hóa tương đương thì chọn mô hình đơn giản hơn (bước 4.4). |
| 20 | Vì sao accuracy 99,9% có thể vô nghĩa? | Class imbalance: 9990 lớp 0, 10 lớp 1; dự đoán toàn lớp 0 vẫn 99,9% nhưng bỏ sót hết lớp 1 (bước 5.2). |
| 21 | Tính cost của M1 và M2 và so với accuracy. | 3910 và 4255; M2 chính xác hơn (90% so với 80%) nhưng đắt hơn (bước 5.3). |
| 22 | Viết công thức precision, recall, F. | TP/(TP+FP); TP/(TP+FN); 2TP/(2TP+FN+FP) (bước 5.4). |
| 23 | Đọc đường ROC: điểm (1,0), (0,0), (1,1), đường chéo, AUC. | Lý tưởng (TPR=1, FPR=0); mọi thứ negative; mọi thứ positive; đoán ngẫu nhiên; AUC 1 lý tưởng, 0,5 ngẫu nhiên (bước 5.6). |
| 24 | Quy trình ba bước dùng siêu tham số? | Train với lưới/greedy; model selection bằng validation rồi dựng lại bằng toàn bộ train; test cuối (bước 6.3). |

| # | Câu hỏi | Đáp án gợi ý |
|---|---|---|
| 25 | Tính khoảng tin cậy 95% cho accuracy 80% trên 100 mẫu. | 0,8 ± 1,96 × √(0,16/100) = [0,722; 0,878] (bước 6.4). |
| 26 | Khi so sánh hơn hai mô hình cần làm gì? | Hiệu chỉnh so sánh bội: Bonferroni hoặc FDR (bước 6.5). |
| 27 | Hai nhóm phương pháp chọn đặc trưng ở DL04a? | Univariate feature importance (chi-squared, information gain) và feature subset selection (black box, tham lam, stepwise backward) (bước 7.1). |

## D. Báo cáo tự kiểm tra tổng

| Hạng mục | Kết quả |
|---|---|
| Đối chiếu nguồn | Đã đọc toàn bộ lớp chữ của DL04a (40 trang), DL04b (64 trang), DL04c (38 trang) và xem ảnh các trang quan trọng: DL04a p6, 7, 8, 16, 20, 22, 25, 26, 28, 31, 32, 35; DL04b p6, 14, 15, 23; DL04c p13, 23, 28, 34. Mọi ví dụ số đã kiểm bằng code hoặc tính tay: cost M1/M2, accuracy, khoảng tin cậy Wald, bảng z, pessimistic 2,5%, đếm điểm trong quả cầu (so với lý thuyết), P/R/F/kappa. |
| Ý tác giả slide và phân tích của mình | Tách bằng nhãn [Mình]. Phần của mình: đáp án vài câu hỏi slide để ngỏ ($2^d$, vì sao không chọn validation cao nhất), bảng P/R/F/κ cho M1/M2, công thức kappa chuẩn, giải thích Bonferroni, bảng so sánh clustering, checklist 7.3, code minh họa. |
| Thông tin bịa | Không thêm thuật toán hay công thức ngoài slide, trừ phần gắn nhãn [Mình]. Ba chỗ kiến thức ngoài slide quan trọng: công thức Cohen kappa, ý nghĩa Bonferroni, công thức xác suất quả cầu trong hộp (chỉ để kiểm chứng). |
| Thiên kiến | Có xu hướng gom thành bảng (mất sắc thái); có xu hướng tự điền đáp án cho câu hỏi slide bỏ ngỏ, nhưng đều gắn nhãn. Slide kiểu 'hard to beat!', 'remarkable factoid' là nhận định của giảng viên, không phải kết luận khoa học. |
| Mức chắc chắn cao | Định nghĩa, công thức, ví dụ số, bảng thuật toán. |
| Mức chắc chắn trung bình | Hình trích từ sách (decision boundary, errors, specific/general); số đọc từ đồ thị (38%, 9%, 32%, 61%, 90%); số train/test của GMM; hình kNN, SVM, DT (chỉ có chú thích). |
| Chưa xác minh | Số dòng của bộ banknotes ('~1300 rows'); công thức paired t-test và cách hiệu chỉnh Bonferroni/FDR (slide chỉ nêu tên); tên tham số sklearn ngoài các tên slide nêu. |